arXiv:2404.10849v1 [cs.RO] 16 Apr 2024

End-To-End Training and Testing Gamification Framework to Learn Human Highway Driving

Satya R. Jaladi[1], Zhimin Chen[1], Narahari R. Malayanur[1], Raja M. Macherla[1], and Bing Li[1]

***Abstract*— The current autonomous stack is well modularized and consists of perception, decision making and control in a handcrafted framework. With the advances in artificial intelligence (AI) and computing resources, researchers have been pushing the development of end-to-end AI for autonomous driving, at least in problems of small searching space such as in highway scenarios, and more and more photorealistic simulation will be critical for efficient learning. In this research, we propose a novel game-based end-to-end learning and testing framework for autonomous vehicle highway driving, by learning from human driving skills. Firstly, we utilize the popular game Grand Theft Auto V (GTA V) to collect highway driving data with our proposed programmable labels. Then, an end-to-end architecture predicts the steering and throttle values that control the vehicle by the image of the game screen. The predicted control values are sent to the game via a virtual controller to keep the vehicle in lane and avoid collisions with other vehicles on the road. The proposed solution is validated in GTA V games, and the results demonstrate the effectiveness of this end-to-end gamification framework for learning human driving skills.**

Index Terms: End-to-End Learning, Gamification Framework, Learn Human Driving, Grand Theft Auto (GTA)

I. Introduction

American drivers as a collective cover nearly 3.2 trillion miles every year and nearly 23% of the driving is done on the highways. A study also shows that Americans spend a total of 71 billion hours every year travelling on the highways [1]. Autonomous driving on highway roads can save a lot of time for people who commute daily and avoid fatal accidents at high speeds. This also applies to vehicles such semis and heavy-duty trucks that are often involved in accidents due to driver error. Highways are also suitable for the implementation of autonomous vehicles due to their well-structured roadways and pedestrian free environment. The current autonomous driving technologies rely a lot on computer vision and AI to solve various problems such as lane detection, traffic sign and pedestrian detection etc.

Deep learning using CNN architecture over the past few years have been playing a major role in the development of computer vision technologies, specifically in the field of object detection and lane detection algorithms. However, in order to perfect these algorithms a significant amount of training and testing is required. Due to the scenario of autonomous vehicles, these cannot be tested in regular environment as it may cause danger to regular pedestrians and vehicles. Building a specific facility is very costly. Apart from this, significant amount of data is required to train these algorithms which would again lean to higher cost and time.

In order to tackle this issue, various simulated environments have been developed which reduce the cost and time required to train and test algorithms. One major problem that many simulated environments are facing is their resemblance to reality or lack thereof. Due to noticeable difference in the visually quality of the simulated world and real-world environment, the trained algorithms cannot be readily deployed into the real world. Here the open world games such as GTA V are developed by games studios with close resemblance to reality, and an intuitive way to decrease the dependency on the real-world data is to utilize data collected in human-driving games [2], [3], [4], [5], [6], [7], [8], [9], [10].

II. Related works

The approach of deep reinforcement learning has also been tested for the development for self-driving vehicles. In this case the agent (the vehicles in our case) gathers the environmental information and tries to maximize its rewards based on a defined policy [11], [12], [13]. In case of self-driving cars, the agent could be rewarded for smooth driving and avoiding colliding with other vehicles and following the traffic rules [14], [15]. The concept of reverse imitation learning has also been studied to train vehicles to perform self driving. Here they have used a path planner algorithm to generate a set of paths and then use inverse reinforcement learning to always select the path that gives agents the highest reward [16], [17], [18], [19]. In this scenario also it requires a simulated environment and large amounts of data for training and testing.

The use of convectional neural network has been tested in off-road driving scenarios by Muller [20], [21], [22]. A driving root Dave is introduced which mapped input images to steering angles. In one publication [23], the authors described a CNN system that detects vehicles and lane markings for highway markings, showing that CNNs are useful in self driving.

Recently studies in navigation [24], [25] have demonstrated the potential of end-to-end mapping image to control. As the network is required to clone the behavior of the training, this is termed as behavior cloning. Instead of using a complicated robotics approach where the vehicle should perform actions like localization, state estimation and navigation [26], [27], [28], [29], [30], the training of autonomous vehicle can be simplified by using the process of behavior cloning, where the neural network learns how to drive the vehicle by observing how a human or a trained agent drives

The authors are with the Department of Automotive Engineering, Clemson University International Center for Automotive Research (CU-ICAR), 4 Research Dr, Greenville, SC 29607, USA.
Corresponding author: Dr. Bing Li, bli4@clemson.edu

the vehicle [31], [32]. Although this approach is not the best case for solving the problem of autonomy, this approach can be used in environments like highways and parking zones. This approach is similar to how a human learns how to drive rather than a robotics approach [33], [34], [35], [36], [37].

The approach of deep learning has also been used for segmented tasks such as obstacle detection as mentioned in [38], [39]. Here they combined the use of deep learning and geometric modeling to detect obstacles in the self driving car's environment. The primary detection is done by the deep learning part but it takes help from the cues coming from the geometric model based on a stereo camera. There has also been several studies that show the current issues and challenges faced while using deep learning to implement scene detection and scene perception [40], [41], [42]. The common theme in most of the issues faced by deep learning is the lack readily available data or high amount of resources required to generate that amount of data.

The evaluation of deep learning techniques to perform object detection in different driving conditions has been studied [43], [44], [45], [46], here the use of transfer learning has been discussed to have an edge while training the neural networks to detect objects in different environments. Similar to the results in our approach transfer learning has produced a significant improvement while training and inference of the networks. The use of virtual data to train a deep learning network and inference it in the real world data has been tested [47], [48], [49], [50]. The use of video games to collect data and augment existing data-sets has been studied here [51], [52]. This helped them increase the size of the data, which is crucial for the training performance and also collect data that is hard to find in real life.

In this work, we investigate whether the network behavior cloning have promising results in game environment. For this purpose, we collected a data-set of GTA highway driving with our programmable labels. The data-set and our configurations will be shared to the community for future game-based driving study and autonomous driving education. Furthermore, we propose a novel game-based learning and testing framework for automated highway driving based on deep learning. Finally, the proposed framework solution is validated in GTA V games, and the results demonstration show the effectiveness of the approaches. Our main contributions are following:

1) We proposed a game-based learning and testing framework for automated highway driving based on deep learning to learn from human driving skills.
2) We validated the framework solution by training and testing an end-to-end network model, and our demonstration show the effectiveness of the approaches.
3) We collected a dataset of GTA highway driving with our programmable labels. The dataset and code are shared to the community for game-based driving study and autonomous driving research.

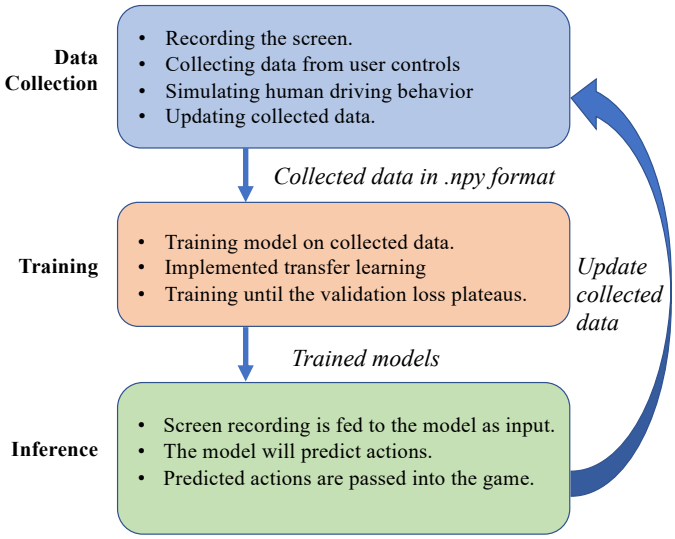

Fig. 1. Framework methodology explaining the methodology implemented to perform data collection, training and inference

III. End-To-End Learning Framework

In order to learn from human driving, we leverage existing gamification solutions such as Grand Theft Auto V (GVA V), under typical highway driving map scenarios. Two neural networks were trained using the same data and tested on a highway environment of GVA-V. The networks are trained using 'end-to-end' learning method as they need to take the input from one end and provide the desired output without any modification in between.

The Nvidia's end-to-end network architecture [25], [53] and VGG-19 architecture were used as our two neural networks of choice. As shown in Fig. 1 the Nvidia's architecture consists of nine layers, including a normalization layer, five convolutional layers, and three fully connected layers. The input images are converted into YUV image format before being passed through the architecture.

The initial layer of the network architecture performs image normalization to reduce the complexity of calculations and is a standard practice. The architecture is then followed by a set of strided and non-strided convolution layers. The main aspect of the convolution layers is to extract various features of varying complexity from the input image. The kernel size for these layers has been empirically chosen by a set of experiments as mentioned in [25], [54], [55]. Then the architecture is followed by set of three fully connected layers. These fully connected layers would help in converting the features extracted from the images to steering and throttle values. The final node from the original architecture has been modified from having just one node that predicts the steering value to two nodes that predict steering and throttle values as it suits out testing purpose.

VGG-19 [56] has been selected to be the second network architecture to train and test on the same data.VGG-19 is primarily used to classify images and it outshined a lot of other state-of-the-art models in 2014 in the ImageNet

challenge. This network architecture contains a total of 16 CNN layers and a kernel size of 3x3 was used in all the convolutional layers, for max-pooling a window of 2x2 was used with a stride of one. Rectified Linear Activation Unit (ReLu) were as the activation function in the entire network. The network is then followed by a set of three fully connected layers and the final node consists of two nodes that output the throttle and steering values.

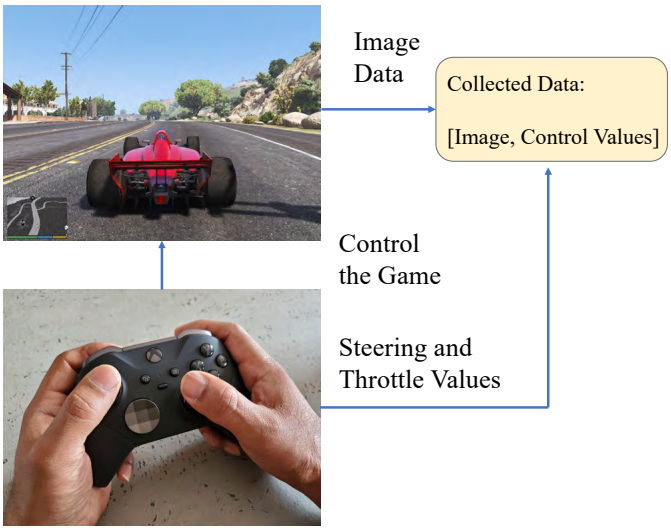

Fig. 2. Data collection system (While the user plays the game, the game footage and control values are simultaneously collected)

In order to avoid training the weights of the entire neural network as it takes a lot of time and resources, the approach of transfer learning has been implemented. In transfer learning approach the VGG-19 model that has been pre-trained with ImageNet dataset is imported directly. The ImageNet dataset contains around 14 million annotated images over which the VGG-19 model has been pre-trained.

During our training phase of this neural network, the weights for the convolution layers are frozen and only the weights of fully connected layers are trained to predict the steering and throttle values. With the help of this approach, the feature recognition aspect of the pre-trained model can be leveraged and helps us overcome the aspect of having lower amount of data to train for feature extraction. The training time required is also significantly decreased as most of the weights of the neural network are already fixed.

Both the models were imported and trained using the TensorFlow 2.0 library in Python. The TensorFlow library makes it easier to load and train the models as the models need not be built from ground-up and the complex calculations required for training are taken care by the TensorFlow functions. Both the networks will be trained on the same collected data. The training procedure and the inference results are discussed in the following sections.

IV. Experiments

In order to collect the required data, the GTA V game is run in windowed mode. The 'Grabscreen' script is used to record the gameplay footage and store the images. The Xbox controller used to control the vehicle is connected to 'Vjoy' software which enables the 'Pygame' Python script to read the commands coming from the Xbox controller.

To simultaneously collect the image and throttle data, both the 'Grabscreen' and 'Pygame' scripts are included in 'CollectData.py' Python script. The throttle values range from -60 to 60 and the steering values range from -100 to 100. The values collected are stored in .npy format which will be then used to train the neural networks. The data collection system is shown in Fig. 2.

A. Data Collection Strategies

The data collected during this process should serve two main purposes, to train the vehicle to stay in the correct lane and to avoid hitting the obstacles/vehicles in front. The data collection strategies implemented to achieve the two-fold purpose are described here. The concept of teaching a neural network how to drive based on how a human being drives is termed as imitation learning.

In this case, an actual person is tasked to drive a vehicle in the highway environment of the game. The corresponding images from the game and control inputs from the person driving the vehicle are recorded simultaneously. The configuration of the controller software is shown in Fig. 3.

First, to teach the vehicle to stay inside the lane, close to 45% of the entire data is collected by driving the vehicle strictly at the center of the lane. This teaches the neural network that when the vehicle is at the center of the lane, the steering input should be close to zero to continue moving in a straight path.

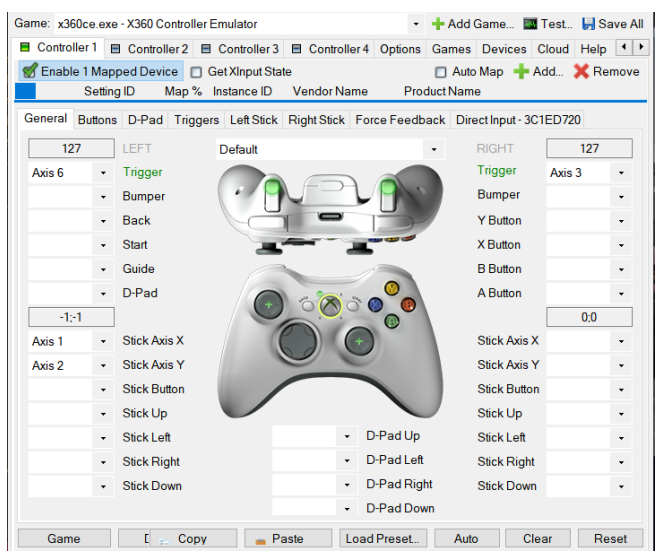

Fig. 3. Virtual controller software (Used to transfer output from neural network to the in-game vehicle)

The next 20% of the data was collected to teach the network what to do when the vehicle is about to cross a lane or deviate from the current lane. During the data collection, the user driving the vehicle lets the vehicle go out of the lane but steers it back as soon as the wheels of the vehicle start to cross the lane marking. This process is done repeatedly to collect enough data that teaches the neural network on how to bring the vehicle back to the center of the lane.

The rest of the data was collected to teach the network to apply brakes i.e., change the throttle value to negative when there's a vehicle in front. The user driving the vehicle repeats the process of applying the brakes when the vehicle is close to another vehicle. These proportion of the three sets of data is later altered based on the inference performance. For example, if the vehicle continues to go out of lane during inference, the proportion of data teaching the vehicle to return to center of the lane is increase and tested again.

Fig. 4. Image cropping (Only a certain section of the screen was fed into the algorithm to avoid redundant data)

B. Image Pre-Processing and Data Augmentation

Image pre-processing and some data augmentation techniques were implemented to clean and improve the data collected. The image was cropped as shown in the Fig. 4 such that it does not contain unnecessary portion of the image that could be the mountains and trees. Random images were selected and were flipped as shown in Fig. 5 to generalize the model and increase the size of the data collected.

The brightness of the images was also adjusted to make the images darker to generalize the training for dark and bright environments. To balance the dataset which contains most of the images with forward throttle and a constant steering angle, the images with steering angle more than 0.3 were doubled and the images with forward throttle were randomly selected and were discarded from the dataset.

C. Training the Networks With Collected Data

The train dataset consisted of $70,000$ images from the GTA V game along with $15,000$ images that were used for validation. Both the models were trained with mean-square error as their error function and Adam as the optimizer. Both the proposed models were trained until the validation loss steadily decreased and care was taken to ensure the network generalized well and did not fit any of the training data. This included early stopping and other regularization techniques like data augmentation, etc. The results of the training process will be discussed in the results section.

Fig. 5. Image left right flipping (Augmenting data by flipping images)

All the models were implemented in TensorFlow 2.0 and were trained on a mobile RTX 2060 with 6GB of video memory. The optimizer used was Adam that regulates the learning rate automatically for each parameter

D. Model Inference

Here the process of how a trained model is implemented in the game environment is demonstrated. To test either of the trained models their ability to control the vehicle in GTA V, the game was set to run in 480p resolution, in a windowed mode. As shown in Fig. 6, the Python inference script was programmed to take screenshots of the image where the GTA V game window is placed.

The Grabscreen package was used to record the screen at where the GTA V window is placed and feed the images into the Python script used for inference. These images were sent to the image processing pipeline which was used for training the neural network in the first place. The images were converted to 200x60-dimension length by width, converted to YUV format and normalized before feeding into the network [57].

The neural network takes the image and predicts the steering and throttle values required to control the vehicle. Fig. 7 shows the exact location in the game where the inference performs.The steering control is multiplied by a gain of 1.5 to improve the steering response and the throttle is capped to 60% to prevent the vehicle moving at high speed. The Vjoy package has been used to act as a virtual Xbox controller to control the vehicle in GTA V game.

The package has been set up to control the vehicle in GTA V and is initialized in the inference script. The predicted values coming from the neural network are sent to the Vjoy object which directly controls the vehicle in GTA V.

The whole system acts as a closed loop control, where the predicted values are sent to control the vehicle and the changes in the game environment are returned by the

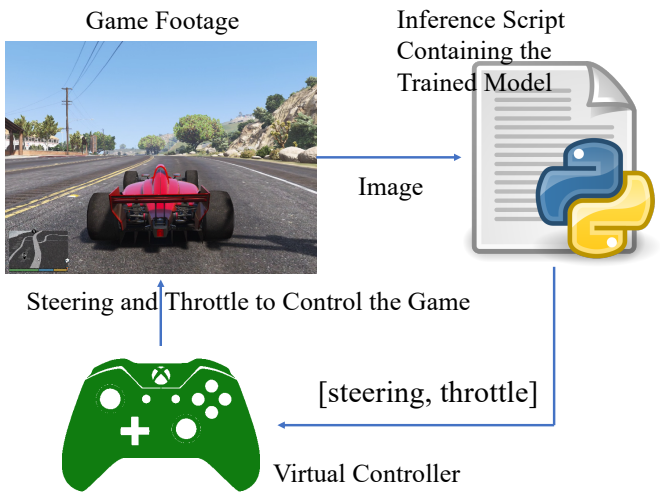

Fig. 6. Inference architecture (Each image frame is sent to the Python script containing the trained architecture. The trained architecture outputs the control values via a virtual controller)

Fig. 7. Location on the GTA V map where the highway data has been collected, and this location has nearly 29 miles stretch of highway available

Grabscreen package which in turn changes the predictions given by the neural network.

V. Results and Discussion

The two selected networks were trained on a mobile RTX 2060 GPU with 6GB of video ram. The networks were trained until the validation loss steadily decreases and the best weights were saved for inference. Out of the two models tested since the VGG-19 model was pre-trained with ImageNet dataset, it converged faster compared to the Nvidia's architecture which were trained from scratch. The total training time for the VGG-19 network was one hour twenty five minutes whereas the Nvidia's network took greater than six hours to train.

As shown in Fig. 8, the VGG-19 provided the least amount of validation loss. The models were trained for 20 epochs

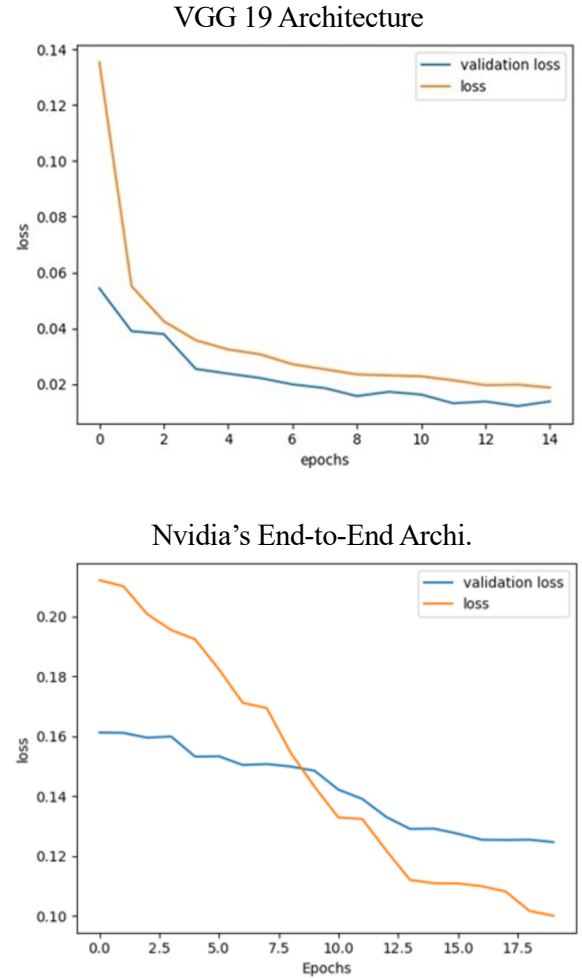

Fig. 8. Training performance comparison of the two model architectures

with a batch size of 128. The training results show that the VGG-19's validation loss is consistently lower than training loss, which implies the model generalized well and did not suffer from overfitting. However, the validation loss for Nvidia's architecture did go higher than the training loss. Compared to VGG-19, it also has higher validation loss at 0.12. Both the models were also implemented to drive the vehicle inside the game the performance is measured.

The trained VGG-19 architecture provided good performance in controlling the vehicle by keeping the vehicle in lane and avoiding the vehicle from colliding with the other vehicle in the highway environment.

Due to the constraints of running both the video game and the inference on the same mobile RTX 2060 GPU, the model is only able to process between 5-10 frames per second. Even though this could negatively affect the performance and response time of the network, it did perform considerably well.

Fig. 9 shows the time driven by the vehicle without completely going out of lane or hitting another vehicle,

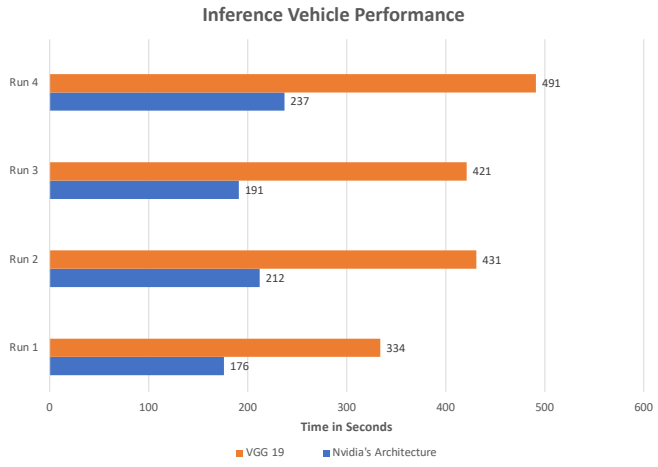

Fig. 9. Inference results (showing how long the vehicle traveled inside the game without completely going out of the lane or hitting another vehicle.)

and the VGG-19 model consistently performed better than the Nvidia's architecture. After each run the training data collected is modified and the models are re-trained again for better performance, hence the overall performance of both the models increased after every run.

As this is an end-to-end architecture, the actions of the vehicle performed during inference cannot be strictly justified, for example we cannot particularly explain why the vehicle was going out of lane in certain scenarios, it is a case of assuming what would have caused the behavior and improvising the training data to negate such undesired instances. In the real world scenarios, CNN networks are primarily used in a modular methodology where their job would only be to detect the lane or obstacles near a vehicle and rest of the control algorithm would be taken care by another control algorithm. Our research result code [1] and comprehensive demonstration video [2] can be seen online.

VI. Conclusion

An end-to-end training and testing gamification framework are proposed in this research, with a CNN model deployed in our framework to learn the human-driving skills in GTA V environment. The vehicle was trained to stay in lane and avoid collisions with the vehicle around it. The performance of the network has been satisfactory. Due to the limits of the GPU performance, the model was limited to running at five frames per second which was one of the reasons why the vehicle was running at slow response time. The future work of this research is to quantify the performance with human subject study in different scenarios with different users and create metric of driving skills learning.

[1] https://github.com/AutoAILab/End2EndDriving
[2] https://www.youtube.com/watch?v=tUFN5y6eTRo

References

[1] W. Kim, V. Añorve, and B. Tefft, "American driving survey, 2014–2017," 2019.

[2] M. Martinez, C. Sitawarin, K. Finch, L. Meincke, A. Yablonski, and A. Kornhauser, "Beyond grand theft auto v for training, testing and enhancing deep learning in self driving cars," *arXiv preprint arXiv:1712.01397*, 2017.

[3] H. Yun and D. Park, "Simulation of self-driving system by implementing digital twin with gta5," in *2021 International Conference on Electronics, Information, and Communication (ICEIC)*. IEEE, 2021, pp. 1–2.

[4] S. Singh, M. Prajapati, N. Vashist, H. S. Rajput, V. Mishra, U. Mittal, and P. Rana, "Action replication in gta5 using posenet architecture with lstm cells," in *2021 2nd International Conference on Intelligent Engineering and Management (ICIEM)*. IEEE, 2021, pp. 544–549.

[5] J. Ru, H. Yu, H. Liu, J. Liu, X. Zhang, and H. Xu, "A bounded near-bottom cruise trajectory planning algorithm for underwater vehicles," *Journal of Marine Science and Engineering*, vol. 11, no. 1, p. 7, 2022.

[6] X. Zhang, H. Liu, L. Xue, X. Li, W. Guo, S. Yu, J. Ru, and H. Xu, "Multi-objective collaborative optimization algorithm for heterogeneous cooperative tasks based on conflict resolution," in *International Conference on Autonomous Unmanned Systems*. Springer, 2021, pp. 2548–2557.

[7] H. Liu, Y. Shen, S. Yu, Z. Gao, and T. Wu, "Deep reinforcement learning for mobile robot path planning," *arXiv preprint arXiv:2404.06974*, 2024.

[8] Y. Chen, W. Huang, S. Zhou, Q. Chen, and Z. Xiong, "Self-supervised neuron segmentation with multi-agent reinforcement learning," in *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*, 2023, pp. 609–617.

[9] Y. Chen, W. Huang, X. Liu, Q. Chen, and Z. Xiong, "Learning multiscale consistency for self-supervised electron microscopy instance segmentation," in *IEEE International Conference on Acoustics, Speech, and Signal Processing*, 2023.

[10] Y. Zhou, X. Li, Q. Wang, and J. Shen, "Visual in-context learning for large vision-language models," *arXiv preprint arXiv:2402.11574*, 2024.

[11] B. R. Kiran, I. Sobh, V. Talpaert, P. Mannion, A. A. Al Sallab, S. Yogamani, and P. Pérez, "Deep reinforcement learning for autonomous driving: A survey," *IEEE Transactions on Intelligent Transportation Systems*, 2021.

[12] Y. Chen, C. Liu, W. Huang, S. Cheng, R. Arcucci, and Z. Xiong, "Generative text-guided 3d vision-language pretraining for unified medical image segmentation," *arXiv preprint arXiv:2306.04811*, 2023.

[13] Y. Zhou, X. Geng, T. Shen, C. Tao, G. Long, J.-G. Lou, and J. Shen, "Thread of thought unraveling chaotic contexts," *arXiv preprint arXiv:2311.08734*, 2023.

[14] H. Yang, "An independent study of reinforcement learning and autonomous driving," *arXiv preprint arXiv:2110.07729*, 2021.

[15] Y. Zhou, T. Shen, X. Geng, C. Tao, C. Xu, G. Long, B. Jiao, and D. Jiang, "Towards robust ranker for text retrieval," in *Findings of the Association for Computational Linguistics: ACL 2023*, 2023, pp. 5387–5401.

[16] T. Phan-Minh, F. Howington, T.-S. Chu, S. U. Lee, M. S. Tomov, N. Li, C. Dicle, S. Findler, F. Suarez-Ruiz, R. Beaudoin, *et al.*, "Driving in real life with inverse reinforcement learning," *arXiv preprint arXiv:2206.03004*, 2022.

[17] J. Wu, Z. Lai, S. Chen, R. Tao, P. Zhao, and N. Hovakimyan, "The new agronomists: Language models are experts in crop management," *arXiv preprint arXiv:2403.19839*, 2024.

[18] Y. Zhou, T. Shen, X. Geng, C. Tao, G. Long, C. Xu, and D. Jiang, "Fine-grained distillation for long document retrieval," *arXiv preprint arXiv:2212.10423*, 2022.

[19] H. A. Arief, M. Arief, G. Zhang, Z. Liu, M. Bhat, U. G. Indahl, H. Tveite, and D. Zhao, "Sane: smart annotation and evaluation tools for point cloud data," *IEEE Access*, vol. 8, pp. 131 848–131 858, 2020.

[20] U. Muller, J. Ben, E. Cosatto, B. Flepp, and Y. Cun, "Off-road obstacle avoidance through end-to-end learning," *Advances in neural information processing systems*, vol. 18, 2005.

[21] S. Li, K. Sun, Z. Lai, X. Wu, F. Qiu, H. Xie, K. Miyata, and H. Li, "Ecnet: Effective controllable text-to-image diffusion models," *arXiv preprint arXiv:2403.18417*, 2024.

[22] W. Ding, S. Li, G. Zhang, X. Lei, and H. Qian, "Vehicle pose and shape estimation through multiple monocular vision," in *2018 IEEE*

International Conference on Robotics and Biomimetics (ROBIO). IEEE, 2018, pp. 709–715.

[23] B. Huval, T. Wang, S. Tandon, J. Kiske, W. Song, J. Pazhayampallil, M. Andriluka, P. Rajpurkar, T. Migimatsu, R. Cheng-Yue, *et al.*, "An empirical evaluation of deep learning on highway driving," *arXiv preprint arXiv:1504.01716*, 2015.

[24] H. Yun and D. Park, "Virtualization of self-driving algorithms by interoperating embedded controllers on a game engine for a digital twining autonomous vehicle," *Electronics*, vol. 10, no. 17, p. 2102, 2021.

[25] M. Bojarski, D. Del Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang, *et al.*, "End to end learning for self-driving cars," *arXiv preprint arXiv:1604.07316*, 2016.

[26] A. Amini, I. Gilitschenski, J. Phillips, J. Moseyko, R. Banerjee, S. Karaman, and D. Rus, "Learning robust control policies for end-to-end autonomous driving from data-driven simulation," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1143–1150, 2020.

[27] A. Prakash, K. Chitta, and A. Geiger, "Multi-modal fusion transformer for end-to-end autonomous driving," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 7077–7087.

[28] J. Yao, C. Li, K. Sun, Y. Cai, H. Li, W. Ouyang, and H. Li, "Ndc-scene: Boost monocular 3d semantic scene completion in normalized device coordinates space," in *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE Computer Society, 2023, pp. 9421–9431.

[29] A. Zhu, J. Li, and C. Lu, "Pseudo view representation learning for monocular rgb-d human pose and shape estimation," *IEEE Signal Processing Letters*, vol. 29, pp. 712–716, 2021.

[30] E. Pérez, C. E. Ardic, O. Çakıroğlu, K. Jacob, S. Kodera, L. Pompa, M. Rachid, H. Wang, Y. Zhou, C. Zimmer, *et al.*, "Integrating ai in nde: Techniques, trends, and further directions," *arXiv preprint arXiv:2404.03449*, 2024.

[31] M.-j. Lee and Y.-g. Ha, "Autonomous driving control using end-to-end deep learning," in *2020 IEEE International Conference on Big Data and Smart Computing (BigComp)*. IEEE, 2020, pp. 470–473.

[32] K. Xu, L. Chen, and S. Wang, "Data-driven kernel subspace clustering with local manifold preservation," in *2022 IEEE International Conference on Data Mining Workshops (ICDMW)*. IEEE, 2022, pp. 876–884.

[33] J. Kocić, N. Jovičić, and V. Drndarević, "An end-to-end deep neural network for autonomous driving designed for embedded automotive platforms," *Sensors*, vol. 19, no. 9, p. 2064, 2019.

[34] C. Liu, S. Lee, S. Varnhagen, and H. E. Tseng, "Path planning for autonomous vehicles using model predictive control," in *2017 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2017, pp. 174–179.

[35] L. Li, "Hierarchical edge aware learning for 3d point cloud," in *Computer Graphics International Conference*. Springer, 2023, pp. 81–92.

[36] ——, "Cpseg: Finer-grained image semantic segmentation via chain-of-thought language prompting," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2024, pp. 513–522.

[37] S. Oehmcke, L. Li, K. Trepekli, J. C. Revenga, T. Nord-Larsen, F. Gieseke, and C. Igel, "Deep point cloud regression for above-ground forest biomass estimation from airborne lidar," *Remote Sensing of Environment*, vol. 302, p. 113968, 2024.

[38] S. Ramos, S. Gehrig, P. Pinggera, U. Franke, and C. Rother, "Detecting unexpected obstacles for self-driving cars: Fusing deep learning and geometric modeling," in *2017 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2017, pp. 1025–1032.

[39] K. Xu, L. Chen, and S. Wang, "A multi-view kernel clustering framework for categorical sequences," *Expert Systems with Applications*, vol. 197, p. 116637, 2022.

[40] A. Gupta, A. Anpalagan, L. Guan, and A. S. Khwaja, "Deep learning for object detection and scene perception in self-driving cars: Survey, challenges, and open issues," *Array*, vol. 10, p. 100057, 2021.

[41] T. Deng, H. Xie, J. Wang, and W. Chen, "Long-term visual simultaneous localization and mapping: Using a bayesian persistence filter-based global map prediction," *IEEE Robotics & Automation Magazine*, vol. 30, no. 1, pp. 36–49, 2023.

[42] Y. Zhou, A. Osman, M. Willms, and A. Kunz, "Semantic wireframe detection."

[43] R. Simhambhatla, K. Okiah, S. Kuchkula, and R. Slater, "Self-driving cars: Evaluation of deep learning techniques for object detection in different driving conditions," *SMU Data Science Review*, vol. 2, no. 1, p. 23, 2019.

[44] T. Deng, G. Shen, T. Qin, J. Wang, W. Zhao, J. Wang, D. Wang, and W. Chen, "Plgslam: Progressive neural scene represenation with local to global bundle adjustment," *arXiv preprint arXiv:2312.09866*, 2023.

[45] Y. Huang and J. J. Yang, "Symmetric contrastive learning for robust fault detection in time-series traffic sensor data," *International Journal of Data Science and Analytics*, pp. 1–15, 2024.

[46] ——, "Semi-supervised multiscale dual-encoding method for faulty traffic data detection," *arXiv preprint arXiv:2212.13596*, 2022.

[47] G. R. Taylor, A. J. Chosak, and P. C. Brewer, "Ovvv: Using virtual worlds to design and evaluate surveillance systems," in *2007 IEEE conference on computer vision and pattern recognition*. IEEE, 2007, pp. 1–8.

[48] T. Deng, Y. Chen, L. Zhang, J. Yang, S. Yuan, D. Wang, and W. Chen, "Compact 3d gaussian splatting for dense visual slam," *arXiv preprint arXiv:2403.11247*, 2024.

[49] T. Deng, S. Liu, X. Wang, Y. Liu, D. Wang, and W. Chen, "Prosgnerf: Progressive dynamic neural scene graph with frequency modulated auto-encoder in urban scenes," *arXiv preprint arXiv:2312.09076*, 2023.

[50] H. Wang, Y. Zhou, E. Perez, and F. Roemer, "Jointly learning selection matrices for transmitters, receivers and fourier coefficients in multichannel imaging," *arXiv preprint arXiv:2402.19023*, 2024.

[51] A. Shafaei, J. J. Little, and M. Schmidt, "Play and learn: Using video games to train computer vision models," *arXiv preprint arXiv:1608.01745*, 2016.

[52] T. Deng, Y. Wang, H. Xie, H. Wang, J. Wang, D. Wang, and W. Chen, "Neslam: Neural implicit mapping and self-supervised feature tracking with depth completion and denoising," *arXiv preprint arXiv:2403.20034*, 2024.

[53] K. W. Tong, P. Z. Sun, E. Q. Wu, C. Wu, and Z. Jiang, "Adaptive cost volume representation for unsupervised high-resolution stereo matching," *IEEE Transactions on Intelligent Vehicles*, vol. 8, no. 1, pp. 912–922, 2022.

[54] K. W. Tong, Z. Shi, G. Zhu, Y. Duan, Y. Hou, E. Q. Wu, and L. Zhu, "Large-scale aerial scene perception based on self-supervised multi-view stereo via cycled generative adversarial network," *Information Fusion*, p. 102399, 2024.

[55] M. Li, Y. Zhou, G. Jiang, T. Deng, Y. Wang, and H. Wang, "Ddn-slam: Real-time dense dynamic neural implicit slam," 2024.

[56] A. Berg, J. Deng, and L. Fei-Fei, "Large scale visual recognition challenge (ilsvrc), 2010," *URL http://www. image-net. org/challenges/LSVRC*, vol. 3, 2010.

[57] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Advances in neural information processing systems*, vol. 25, 2012.